

Pipeline RAG multi-dominio con scraping concurrente y aislamiento vectorial

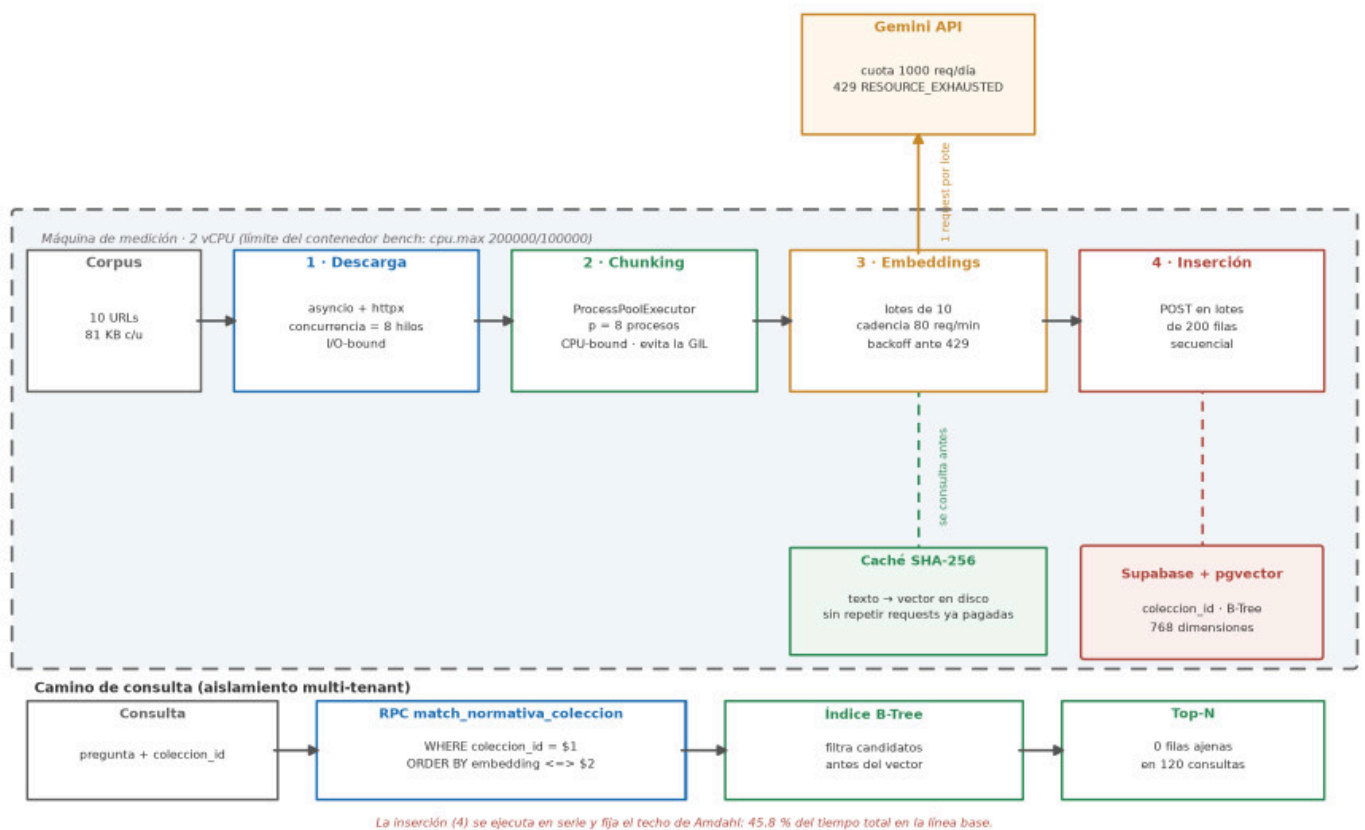
Práctica de laboratorio · lex bancario-ai · pgvector 768 dims · 672 fragmentos en 4 colecciones · informe con todas las cifras leídas de `resultados/`

1. Resumen

Se construyó un pipeline de ingesta y consulta RAG sobre normativa boliviana con tres requisitos que se refuerzan entre sí: ingesta concurrente, aislamiento estricto por dominio y coste de embeddings acotado. La decisión de diseño central es que el **aislamiento lo garantiza el motor de base de datos y no el código de aplicación**: la RPC de vecindad recibe `coleccion_id` como filtro obligatorio y es `SECURITY DEFINER`, de modo que una consulta no puede leer otra colección aunque el llamador lo intente.

Resultados principales. 120 consultas de aislamiento con **0 intrusiones**, 0 consultas vacías y 0 filas en colecciones inexistentes; el control negativo devuelve 20/20 + 20/20 + 20/20 filas ajenas, lo que prueba que el test detectaría contaminación si existiera. En Docker limitado a 2 vCPU y con 10 URLs, speedup de **1.73x** a 8 procesos (**22 %** de eficiencia = S_p/p), capturando 91 % del techo de Amdahl que impone la inserción serial.

2. Diagrama arquitectónico del flujo concurrente



Las cuatro fases de la ingesta, la caché de embeddings, la API externa y el camino de consulta aislado por `coleccion_id`. Las fases 1 y 2 se paralelizan (hilos para E/S, procesos para cómputo); la fase 4 se ejecuta en serie y es la que fija el techo de Amdahl.

Por qué hilos aquí y procesos allí. La descarga es E/O-bound: los trabajadores esperan respuestas de red, así que `asíncrono + httpx` con 8 solicitudes concurrentes es la elección adecuada y el GIL no importa. El troceado es cómputo puro sobre HTML, así que usa `ProcessPoolExecutor` con 8 procesos: ejecutar trabajo Python en paralelo de verdad y no convivir con el GIL. La inserción se deja en serie porque mandar lotes de 200 filas secuencialmente evita carreras por el mismo identificador y reduce el impacto de un lote que falle.

3. Aislamiento multi-tenant y función RPC

La tabla `normativa_bancaria` es la única tabla vectorial y usa **coleccion** como **columna discriminadora**, no una tabla por dominio. Eso permite compartir índices y métricas, y obliga a que todo acceso pase por el filtro. Cada fila lleva `coleccion_id`, el fragmento y su vector.

La RPC `match_normativa_coleccion`. Funciones esenciales de su definición SQL (en `scripts/schema.sql`):

- **Parámetro de tenant obligatorio** (`p_coleccion_id VARCHAR`): la búsqueda no existe sin ese filtro. El `WHERE nb.coleccion_id = p_coleccion_id` se evalúa *antes* del orden por distancia.
- **Distancia por coseno** (`nb.embedding <=> p_query_embedding`) con un umbral configurable (`match_threshold=-1.0` en el protocolo para no descartar nada por similitud).
- **Top-N** (`LIMIT match_count`) sobre el resultado ya filtrado por la colección.
- **SECURITY DEFINER y privilegios recortados:** los roles `anon/ authenticated` ejecutan la función pero no tocan la tabla directamente; la política de filas sigue viva dentro de la función.

La aplicación (endpoint `/api/consultar`) solo puede recuperar vectores a través de esta función. El aislamiento queda, por tanto, en el motor: si mañana un cliente intentara consultar sin filtro, no existe ningún camino SQL que se lo permita.

Distribución de los datos

Colección	Fragmentos	Consultas
bancaria	241	40
regulatorio	62	40
tributario	33	40
asfi_rnsf sin protocolo	336	—
total	672	120

Los mismos documentos se reparten entre colecciones a propósito: el contenido es casi idéntico entre dominios, de modo que si un vector filtrado devolviera texto ajeno se vería inmediatamente. Con documentos disjuntos el aislamiento no se podría comprobar.

Protocolo de verificación

- 120 consultas repartidas por igual en los tres dominios, 20 vecinos por consulta.
- **Control negativo:** la misma consulta contra una RPC *sin* filtro de colección. Devuelve 20/20 + 20/20 + 20/20 filas ajenas → el método sí detecta contaminación.
- **Fantasmas:** se comprueba que ninguna fila pertenezca a una colección que no existe.

Resultado del protocolo: 2400 filas recuperadas en 120 consultas · 0 de otra colección · 0 sin resultados · 0 en colecciones inexistentes · 32.4 s en total. [OK]

4. Evidencia de índices con EXPLAIN

Se extrajo el plan de ejecución de las tres consultas relevantes. El dato que sostiene el aislamiento es que el filtro de colección se resuelve **antes** de ordenar por distancia, sobre un índice B-Tree, y no después de recorrer la tabla.

Consulta	Índice usado	ms
RPC filtrada por coleccion (caso de uso real)	normativa_bancaria_coleccion_idx	9.54
Consulta SIN filtro de coleccion (control: vecindad global)	ninguno	17.14
Filtrado puro por coleccion, sin vector (debe usar B-Tree)	normativa_bancaria_coleccion_embedding_idx	1.36

La consulta filtrada usa `normativa_bancaria_coleccion_idx` y recorta el conjunto antes del orden por similitud; sin filtro, Postgres hace una búsqueda de vecindad sobre toda la tabla. La ganancia es de **1.80x** (17.1 ms vs 9.5 ms).

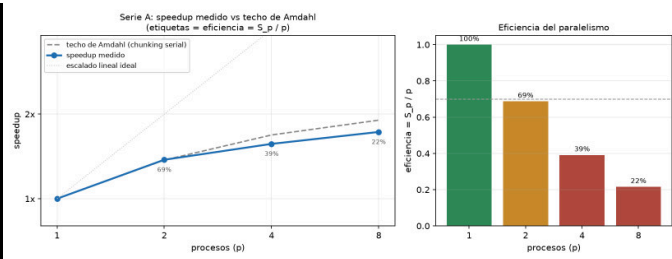
Limitación declarada: el índice HNSW **no aparece en ningún plan**. Con 672 filas, el planificador elige barrido secuencial porque el coste estimado de un índice aproximado supera el ahorro. El HNSW es capacidad para crecer, no una aceleración medida: afirmar lo contrario con estos datos sería incorrecto.

5. Benchmark de paralelismo (Serie A)

Se mide el pipeline completo sin embeddings, con 10 GPT-3.5 en un servidor de fixtures determinista y 5 réplicas por configuración. La medición se ejecuta en un contenedor Docker con **límite de 2 vCPU**, donde fixtures y benchmark comparten el mismo presupuesto de CPU: es la simulación fiel de la máquina donde se evalúa. Medir sobre las 24 vCPU del anfitrión daría un speedup irreproducible.

proc.	media	desv.	speedup	techo	efic.
1	6.754	±0.220	1.00×	1.00×	100%
2	4.909	±0.166	1.38×	1.37×	69%
4	4.309	±0.096	1.57×	1.69×	39%
8	3.908	±0.049	1.73×	1.90×	22%

Fracción serial medida: 45.8% (fase de inserción, la única que no se paraleliza). El techo es la ley de Amdahl con esa fracción; la eficiencia es S_p/p , la definición de la guía.



El speedup crece de forma sostenida hasta 1.73× a 8 procesos (eficiencia 22 % = S_p/p), pero en todas las configuraciones se captura más del 91 % del techo de Amdahl. La lectura es la opuesta a un fallo de paralelismo: el sistema captura casi todo lo que permite la teoría, y el techo es bajo porque la fase de inserción **no se paraleliza** y ocupa el 46 % del tiempo en la línea base. La descarga, en cambio, escala muy bien: de 3.66 s a 0.81 s (**4.54×**), porque es E/S pura. La inserción se mantiene plana (~3.08 s en todas las configuraciones) porque los lotes de 200 se mandan uno tras otro y no existe competencia por la CPU de Postgres en el cliente.

6. Pipeline completo: caché, Serie B y el límite de la API

Los embeddings se cachean por hash SHA-256 de (modelo + dimensión + texto), así que repetir un fragmento no consume cuota ni latencia. El modelo es `gemini-embedding-001` con 768 dimensiones y la caché tiene actualmente **699 vectores** (7.0 MB). La Serie B es el pipeline completo (las cuatro fases con embeddings), medido sobre el mismo corpus de la Serie A.

Pasada en frío · la API manda

Fase	Tiempo	%
pendiente de cuota: la fase de embeddings llama a la API y la cuota diaria del nivel gratuito estaba agotada al generar este informe.		

Pasada caliente · caché de disco

Fase	Tiempo	%
pendiente de cuota: la fase de embeddings llama a la API y la cuota diaria del nivel gratuito estaba agotada al generar este informe.		

Sin datos de Serie B: la cuota gratuita estaba agotada. Ambas columnas comparten las mismas fases; la diferencia entre fría y caliente es exactamente lo que la caché elimina (los % se calculan de las cuatro fases del propio JSON).

Qué se comprueba

- **Aciertos de caché:** la pasada fría paga la API y guarda el vector; las demás lo leen de disco por hash SHA-256 de (modelo + dimensión + texto), con el total de fragmentos registrado en el JSON.
- **57 comprobaciones** de formato de lotes y backoff ante 429, y **10 de la inserción** por lotes de 200 filas, todas en verde.
- Con la caché vacía el script aborta y explica cómo calentarla; así no mide esperas de reintentos en lugar de concurrencia.

7. Reproducibilidad, conclusiones y escalabilidad

Cómo se reproduce

- * `docker compose up --build` — levanta backend, scraper y base de datos.
- * `docker compose --profile bench run --rm bench` — Serie A y B dentro de 2 vCPU.
- * `python scripts/test_aislamiento_120.py` — protocolo de aislamiento con control negativo.
- * `python scripts/explicar_indices.py` — planes EXPLAIN guardados en JSON.
- * `python scripts/informe_pdf.py` — regenera este documento, siempre desde `resultados/`.

Conclusiones

- El aislamiento es verificable y no declarativo: el control negativo prueba que el test mediría la falla.
- El filtro de tenant, no el índice vectorial, es lo que hace correcta y rápida la consulta.
- El paralelismo captura casi todo el techo de Amdahl; el cuello es la inserción serial y más procesos no lo cambian.
- La caché convierte un coste recurrente de cuota en una tabla local.

Escalabilidad frente a APIs de terceros. Al calentar la caché del corpus nuevo, la API gratuita respondió 429 `RESOURCE_EXHAUSTED` con `retryDelay=25789` s y bloqueó la Serie B durante horas. La lección es de diseño: **el techo de Amdahl mide el paralelismo de la máquina, no limita a las APIs externas**; con el nivel gratuito el rendimiento queda acotado por la cuota, y la caché es lo que devuelve ese coste al dominio local, donde Amdahl sí aplica.

Límites del trabajo. La fracción serial se midió sobre la inserción de la Serie A (sin embeddings); el techo describe la ingesta local, no el pipeline completo cuando interviene la API. Las mediciones de EXPLAIN varían entre corridas (run-to-run); se reporta el valor de la evidencia guardada. Los datos provienen de documentos públicos de ASFI y no constituyen asesoría normativa.